

StackWeft: Field-Flow Compilation for Token-Efficient Full-Stack Software Delivery

Kailing Wang

School of Computing and Data Science, The University of Hong Kong

wk1151@connect.hku.hk

Abstract

Large language model coding agents can modify real software repositories, but their session-centric interaction pattern repeatedly charges for rediscovering project structure. This paper studies a narrower but common workload: translating product-manager style natural-language requests into verified full-stack changes. We present StackWeft, an orchestration system that compiles a request into a repository-specific *Field Flow Graph*: typed slots anchored on an existing “shadow” field, deterministic evidence checks for each slot, and counterexample sentinel probes that must reach the backend model, API payload, and frontend DOM. Generation becomes bounded slot filling with recipe replay, deterministic templates, and LLM fallback. Across nine verified Conduit/RealWorld tasks under held-constant GLM-5.1, Claude Sonnet 4.6, and DeepSeek-V4-Pro routes, StackWeft passes 9/9 tasks while a session-based agent baseline passes 7/9; StackWeft uses 60.3k tokens vs. 972.1k, a $16.1\times$ reduction. On a different React/Node/PostgreSQL CRUD repository, warm recipe reuse reduces token use from the session agent’s 158.0k to 5.6k while preserving cross-stack coverage. A six-route ablation (GLM, Claude Sonnet 4.6, DeepSeek, Kimi, Codex, MiniMax) shows that the compiled workflow transfers across heterogeneous model APIs. The results suggest that recurring software-delivery work benefits from compiling cross-stack structure into reusable, verifiable contracts rather than delegating each request to an unconstrained session.

1 Introduction

Modern coding agents combine code-specialized language models with filesystem tools, shell commands, and iterative repair loops [Chen et al., 2021, Li et al., 2022]. They are flexible enough to resolve real issues in benchmarked repositories [Jimenez et al., 2024, Yang et al., 2024], but flexibility is not free. A session-based agent commonly re-discovers project structure, repeatedly reads the same files, and feeds long interaction histories back into the model. We call this the *session tax*: each new product request starts cold unless a human manually preserves and curates context. For product-driven full-stack changes—“add a subtitle to articles, make it editable, and show it on the list and detail pages”—this is wasteful. The task is not arbitrary search; it is a structured program transformation across frontend, backend, and database layers.

StackWeft asks whether such repeated structure can be compiled before the model writes code. Its core observation is that additive full-stack changes often have a *shadow field*: an existing field already wired through the repository. If the system can locate that field’s model declaration, controller write path, frontend service payload, form state, list render, and detail render, then a new field can be delivered by filling a finite set of repository-specific slots. The model no longer needs to carry a whole-agent transcript; it receives one small file window, one slot instruction, and a machine-checkable evidence contract. Repeated work is stored as a recipe in a database, not as fragile conversational memory. This matches the practical product goal: the system should become cheaper and faster on recurring delivery patterns without asking the user to manage sessions.

This paper makes three contributions:

- We introduce Field Flow Graphs, a repository-profiled representation for additive full-stack field changes with typed slots, anchors, and per-slot evidence.
- We describe a bounded delivery pipeline that combines deterministic profiling, recipe replay, slot-local LLM generation, and sentinel-probe verification.
- We evaluate StackWeft against a session-based agent baseline and across multiple model backends, showing large token reductions at equal verification success on the studied workload.

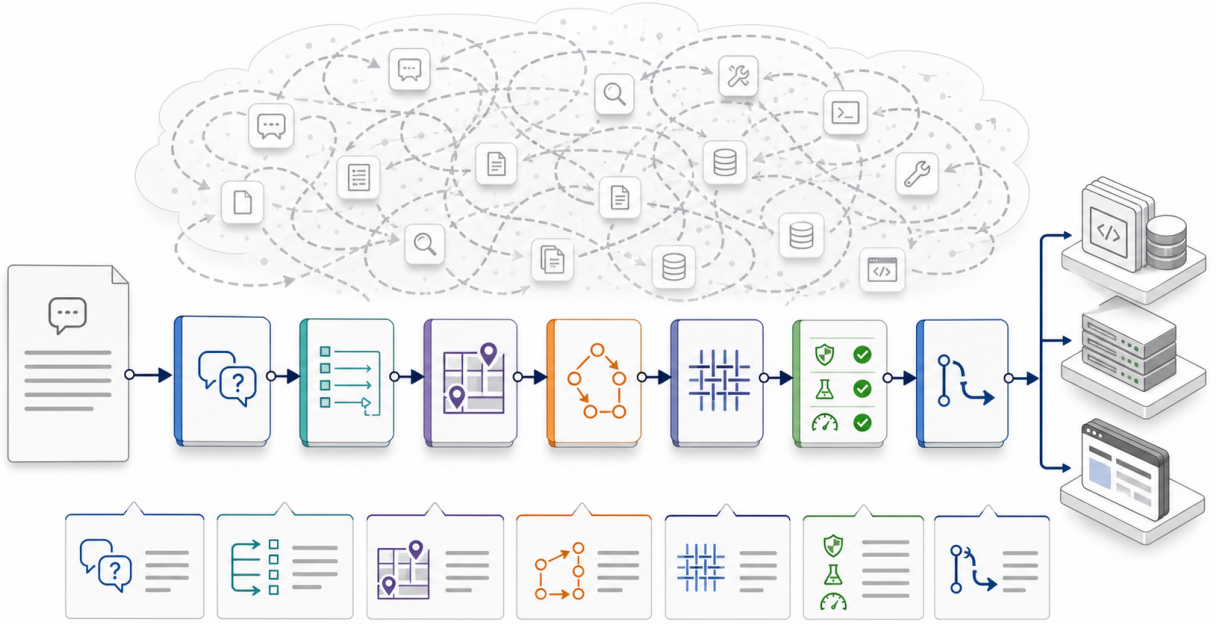


Figure 1: **StackWeft compiles software delivery into verified slot filling.** A product request is clarified, planned, localized, compiled into a Field Flow Graph, woven into code, and checked by tests and cross-stack sentinel probes. For contrast, the cloud above depicts the same request handled by a session-based agent loop: repeated context discovery and tool use that leave behind no reusable typed contract.

2 Related Work

Code models and tool-using agents. Large language models trained on code made repository-scale generation practical [Chen et al., 2021, Li et al., 2022]. ReAct [Yao et al., 2023] and Toolformer [Schick et al., 2023] established patterns for interleaving language-model reasoning with external actions, while Reflexion [Shinn et al., 2023] and Self-Refine [Madaan et al., 2023] add feedback loops. StackWeft uses the same premise—models benefit from tools—but changes the unit of work from an unconstrained trajectory to a compiled slot contract.

LLM software engineering agents. SWE-bench [Jimenez et al., 2024] and systems such as SWE-agent [Yang et al., 2024], AutoCodeRover [Zhang et al., 2024], OpenHands [Wang et al., 2024], and multi-agent software organizations such as MetaGPT [Hong et al., 2024] have made real-repository software repair and delivery primary targets. Agentless [Xia et al., 2024] argues that careful localization and patch generation can outperform complex agent scaffolds on some tasks. StackWeft is aligned with this shift away from maximal autonomy: it specializes further on full-stack product-delivery patterns and verifies value flow through application-specific probes.

Structured code transformation and program sketching. Pattern-driven program transformation predates language models. Coccinelle applies one abstract *semantic patch* across many concrete sites in Linux device drivers [Padioleau et al., 2008], and Comby generalizes lightweight template-based structural rewriting across languages [van Tonder and Le Goues, 2019]; industrial codemod frameworks follow the same recipe. Program sketching completes holes in a partial program subject to a correctness specification [Solar-Lezama et al., 2006], and partial evaluation specializes a program against its static inputs [Futamura, 1999, Jones et al., 1993]. StackWeft inherits this lineage: a Field Flow Graph is a repository-specific sketch whose holes are slots and whose specification is executable evidence. The differences are that the sketch is derived automatically by profiling a shadow field rather than authored by a human for each change pattern, and that holes may be discharged by an LLM when deterministic rewriting runs out.

Compiled LM workflows and memory. Retrieval-augmented generation [Lewis et al., 2020] reduces the need to place all knowledge in a prompt, and systems such as DSPy [Khattab et al., 2024] and LMQL [Beurer-Kellner et al., 2023] treat language-model calls as programmable components rather than opaque chat sessions. Agent-memory systems such as MemGPT page conversational state in and out of a bounded context [Packer et al., 2023]. StackWeft makes a similar compilation move for software delivery, but with a stricter trust model: persistent memory is a typed

recipe and field-flow graph, not an ever-growing conversation, and a memory entry is consulted only if its evidence re-validates against the current working tree.

Program repair and verification. Automated repair systems such as GenProg [Le Goues et al., 2012] and Prophet [Long and Rinard, 2016], and surveys of repair [Gazzola et al., 2019], emphasize that generated patches need executable or semantic checks. Rather than asking an LLM to infer all invariants in one long context, StackWeft derives a small contract from an existing field and checks it with deterministic evidence plus runnable sentinel tests.

3 StackWeft

3.1 Problem Framing

StackWeft targets a recurring class of software-delivery requests: additive full-stack field changes. A user describes a desired product change in natural language, such as adding an editable article subtitle and rendering it in both list and detail views. The desired output is not a free-form patch; it is a cross-stack value-flow transformation. The new value must be represented in the backend schema, accepted by create/update paths, serialized through the API, sent by the frontend write service, stored in editor state, and rendered in user interfaces.

We formalize this as follows. Given a repository $\mathcal{R}$, a request r , a new field f , and a shadow field h that already traverses the relevant stack, produce a patch Δ such that:

$$\mathcal{R} + \Delta \models C_{\text{model}}(f) \wedge C_{\text{write}}(f) \wedge C_{\text{payload}}(f) \wedge C_{\text{render}}(f) \wedge C_{\text{probe}}(f).$$

The C terms are executable contracts rather than informal desiderata. For example, C_{write} may require both create and update assignments, and C_{probe} requires a generated sentinel value to flow through the same paths that a real user value would take. The optimization target is not only correctness, but also the amount of model context needed to reach correctness. A general coding agent treats the whole repository as a search space; StackWeft instead specializes the task against the repository before generation, in the spirit of partial evaluation [Futamura, 1999, Jones et al., 1993]: structure that is static given the repository—paths, anchors, evidence—is resolved at compile time, and only the residual uncertainty is left as slots to fill.

This framing is the central design decision. StackWeft does not ask the model to “understand the app” in one long session. Instead, it turns repeated cross-stack structure into a typed intermediate representation. Once the repository has been profiled, subsequent product requests reuse that structure without re-reading the same files into a chat transcript.

3.2 From Request to Compiler Pipeline

Given a natural-language requirement r and repository $\mathcal{R}$, StackWeft runs a staged pipeline:

$$r \rightarrow \text{CLARIFY} \rightarrow \text{PLAN} \rightarrow \text{LOCALIZE} \rightarrow \text{COMPILE} \rightarrow \text{GENERATE} \rightarrow \text{VERIFY}.$$

CLARIFY extracts the intended entity, field name, display kind, and ambiguities that must be resolved before editing. PLAN decides whether the request belongs to the additive field-flow family or should fall back to a general agent path. LOCALIZE maps repository-specific surfaces: model files, migrations, controllers or services, frontend request builders, forms, and render sites. COMPILE turns those surfaces into a Field Flow Graph. GENERATE executes the graph slot by slot. VERIFY runs both ordinary tests and cross-stack sentinel probes.

The important boundary is between COMPILE and GENERATE. Before that boundary, StackWeft is mostly deterministic: it searches, profiles, and constructs contracts. After that boundary, each model call is local: it receives one slot, one anchor, and one evidence contract. This separates structural discovery from code generation, which is why the same model can spend fewer tokens without being intrinsically stronger.

3.3 Field Flow Graphs

The Field Flow Graph is StackWeft’s intermediate representation. It captures where a value must appear, how each edit should be anchored, and how the system will know that the edit is semantically present. A graph is a set of slot nodes:

$$G = \{s_i = (p_i, a_i, \tau_i, I_i, E_i, D_i)\}_{i=1}^n,$$

where p_i is a repository path, a_i is an anchor derived from the shadow field h , τ_i is the slot type, I_i is a slot-local instruction, E_i is a set of evidence predicates, and D_i lists predecessor slots whose contracts must hold before s_i is attempted. The dependencies make G a directed acyclic graph that mirrors the value flow: schema slots precede write-path slots, which precede payload, frontend, and render slots, and probe slots depend on everything they observe.

Table 1: Typical Field Flow Graph slots for an additive article-field delivery. A different repository may use different paths, but the slot roles and evidence types remain stable.

Slot role	Example surface	Evidence sketch
Backend schema	Sequelize model, migration	field attribute and SQL type exist
Write path	create/update controller	request, create, and update assignments exist
API payload	serializer or response object	field returned to client
Frontend write	request service	field sent in payload
Editor state	form state and input	field editable by user
Render sites	list card, detail page	value rendered in DOM
Probes	generated tests	sentinel reaches model, payload, and DOM

GENERATE fills slots in a topological order of D (Algorithm 1), so an evidence failure surfaces at the earliest layer that can explain it rather than as a distant downstream symptom. The graph is not a data-flow analysis in the compiler-theory sense; it is a delivery contract for a product-level value flow.

Anchors. A slot anchor is usually a shadow-field pattern rather than a line number. If `description` is already wired through the stack, the new `subtitle` field can be inserted near the same declaration, request destructure, payload object, or JSX render block. Anchors make the graph robust to small formatting changes and avoid asking the model to rediscover local style.

Types. Slot types encode the expected edit shape. A schema slot uses a storage type such as string, text, boolean, image URL, or indexed slug. A render slot uses a display type, e.g. paragraph text, image tag, badge, or link. These types are separate because a product-level kind can map to different code surfaces: an image URL is a database string but a frontend image render.

Evidence. Evidence is deliberately executable. A simple slot may require a regex such as `{field}` to appear at least once. More precise slots declare structured checks: a request destructure, a create-object pair, and an update assignment must all exist. Formally, each $e \in E_i$ is a decidable predicate over the working tree, and a candidate patch δ is accepted for s_i iff $\bigwedge_{e \in E_i} e(\mathcal{R} + \delta)$ holds; there is no acceptance path that bypasses evidence. The engine is agnostic to repository shape; it only evaluates evidence declared by the profiled skill.

3.4 Repository Profiling and Skill Specialization

A Field Flow Graph is repository-specific, but the compiler should not hardcode one repository. StackWeft therefore separates a general field-flow skill from a repository profile. The profile records discovered roots, entity names, model style, table names, frontend component surfaces, and known shadow fields. The skill declares roles and evidence templates; the profile binds those roles to actual files. Compilation is thus a binding step: the skill maps each slot role to a surface query, an anchor template, and an evidence template; the profile is the environment that resolves surface queries to concrete paths and shadow anchors; and G is the image of the skill under the profile.

This separation is crucial for generality. Conduit uses a React/Express/Sequelize layout with frontend and backend roots. The secondary CRUD repository uses server and webclient roots, object-form Sequelize attributes, and reducer state. In both cases, the compiler emits the same logical slot roles while the profile supplies different paths and anchors. Thus the reusable artifact is not a repository-specific patch; it is a typed recipe for a family of edits.

3.5 Recipe Memory

StackWeft’s persistent memory is a recipe, not a transcript. A recipe is a tuple $(\rho, \tau, \kappa, \delta[\cdot], E)$: a slot role ρ , a slot type τ , an anchor context κ keyed by a hash of the code surface it applies to, a patch template $\delta[\cdot]$ parameterized by field name and display kind, and the evidence E that accepted its original instantiation. Replay is substitution followed by revalidation: instantiate $\delta[f']$ for the new field, apply it to the current repository, and immediately re-run the slot evidence. If the evidence fails, the recipe is discarded for that slot and the system falls back to templates or model generation; a stale recipe can therefore cost a cache miss, never a wrongly accepted edit.

This design gives memory a falsifiable interface. Conversation memory is cheap to append but hard to trust: it may contain stale paths, irrelevant requirements, or hidden assumptions. Recipe memory is useful only when it can recreate a patch that passes the current evidence contract. That is why repeated field-flow tasks can become cheaper without weakening verification.

Algorithm 1 Evidence-gated weaving of a Field Flow Graph.

Require: graph G with dependencies D ; strategy ladder $\Sigma = \langle \text{REPLAY}, \text{TEMPLATE}, \text{LLM} \rangle$; retry budgets k_σ

```

1: function FILL( $s_i = (p_i, a_i, \tau_i, I_i, E_i, D_i)$ )
2:   for  $\sigma \in \Sigma$  applicable to  $\tau_i$  do
3:     for  $t = 1, \dots, k_\sigma$  do
4:        $\delta \leftarrow \sigma(p_i, a_i, I_i)$  ▷ proposal sees one file window only
5:       apply  $\delta$  to the working tree
6:       if  $\bigwedge_{e \in E_i} e(\mathcal{R} + \delta)$  then
7:         generalize  $\delta$  into a recipe; return accepted
8:       end if
9:       revert  $\delta$ ; append failed predicates to  $I_i$  ▷ rollback is unconditional
10:    end for
11:  end for
12:  return unsatisfied
13: end function
14: for  $s_i$  in a topological order of  $(G, D)$  do
15:   if FILL( $s_i$ ) = unsatisfied then
16:     abort with contract  $(s_i, E_i)$  ▷ fail fast: no silent partial delivery
17:   end if
18: end for
19: run probe slots and the repository test suite; on failure, localize to the owning slot and re-FILL( $\cdot$ ) only that slot

```

3.6 Generation by Degradation

StackWeft fills each slot by descending through increasingly general mechanisms:

1. **Recipe replay.** If a prior successful edit exists for the same slot surface, replay a parameterized patch and re-check evidence.
2. **Deterministic templates.** For low-entropy slots such as migrations and simple render insertions, instantiate templates.
3. **Slot-local LLM patching.** Otherwise, show the model only the relevant file window, field name, shadow field, instruction, and evidence.

Failed evidence rolls back the attempted edit and triggers a targeted retry. Thus, even when an LLM is used, it edits one bounded surface rather than navigating the full repository. Algorithm 1 gives the weaving loop that ties the graph, the ladder, and the evidence gate together.

The degradation order is important. It encodes an engineering preference: deterministic reuse should win over language-model generation whenever the contract is already known. The model is reserved for residual uncertainty, such as adapting a render insertion to an unfamiliar component shape. This is the mechanism behind the token reduction: StackWeft does not merely compress a large prompt; it avoids constructing one.

Each model prompt is intentionally narrow. It contains the requirement summary, the slot role, the target file window, the shadow anchor, the new field metadata, and the evidence predicates. It does not contain the full repository, prior unrelated turns, or all other slots. Once the patch is proposed, the filesystem is the source of truth: StackWeft evaluates evidence on disk and either accepts, rolls back, or retries the slot.

This bounds cost structurally. Let $L \subseteq G$ be the slots whose ladder reaches the LLM rung, W_i the file window shown for slot s_i , and c_0 the fixed cost of the staged pipeline (clarify, plan, localize). Total billed input is

$$T_{\text{in}} = c_0 + \sum_{s_i \in L} (|W_i| + |I_i| + |E_i|),$$

linear in $|L|$ with constant-bounded windows and independent of how many requests preceded this one. A session agent with an append-only transcript re-submits its history h_t at step t , so its billed input grows as $\sum_t |h_t| = \Theta(T^2)$ in the number of tool steps T even when each step appends a constant amount. The benchmark ledgers in Section 5 are consistent with this asymmetry: the baseline’s cost is dominated by re-sent input tokens, while StackWeft’s per-task cost stays in a narrow band.

3.7 Verification

The verification gate combines differential linting, runnable test suites, and cross-stack evidence. For Conduit, StackWeft also writes counterexample sentinel tests: a unique probe value must be defined in the backend model, included in the frontend write payload, and rendered in the list/detail DOM. Passing requires the value to flow through the stack, not merely for the field name to appear in a diff. Each probe value is a generated nonce v chosen to occur nowhere in the repository before the change; uniqueness makes the check conclusive, because any later occurrence of v in the API payload or rendered DOM must have flowed through the new field rather than coincided with existing text.

The sentinel is a counterexample to shallow success. A patch that adds `subtitle` to a model file but forgets update persistence may satisfy a grep-based check, yet it will fail a probe that edits an article and expects the value to reappear. Likewise, a patch that sends the value in the frontend payload but never renders it fails the DOM probe. The verification layer therefore converts a product requirement into observable behavior.

StackWeft uses three levels of acceptance. Slot evidence accepts local edits. Cross-stack evidence accepts value-threading across files. Runnable tests accept behavior under the repository’s own test runner. If any level fails, the system does not continue accumulating context; it localizes the failing slot or probe and retries only that surface. This is another difference from a session agent: the repair target is a contract node, not the whole conversation state.

3.8 Design Invariants

The pipeline is held together by five invariants that we treat as design contracts rather than implementation details:

1. **Filesystem as ground truth.** No acceptance decision depends on a model’s self-report; every evidence predicate is evaluated against the working tree.
2. **Transactional edits.** A patch either satisfies its slot evidence or is reverted before the next attempt; later slots never observe an unverified intermediate state.
3. **Slot-local prompts.** A model call’s context is a function of the requirement, the profile, and one slot—never of other slots’ transcripts—so per-call input is bounded by the file window.
4. **Falsifiable memory.** A recipe contributes only if its evidence re-validates on the current tree; stale memory degrades to a cache miss, not to a wrong edit.
5. **Determinism first.** Mechanisms are ordered by decreasing determinism, and the model is consulted only for residual uncertainty that replay and templates cannot discharge.

Together, these invariants compile away avoidable agency. A general agent must decide what to inspect, what to remember, and how to verify after every tool call. StackWeft moves those decisions into an explicit representation: slots determine what must be edited, anchors determine where edits should land, evidence determines whether edits are acceptable, and recipes determine what can be reused. The model remains useful, but it is no longer responsible for the entire control loop.

This representation also explains the empirical claims. The main benchmark does not show that StackWeft has a better model than the session agent; the model route is held constant. It shows that a compiled workflow can ask a smaller question of the same model. The route robustness experiment then asks the complementary question: once the workflow is compiled, does it depend on a single model’s style? The answer is no for the studied task family, because the prompt surface is slot-local and the acceptance criteria are external.

4 Experimental Setup

Primary repository. We evaluate on Conduit, a React/Express/Sequelize RealWorld monorepo [RealWorld, 2026, TonyMckes, 2026]. All deliveries start from the same scaffold commit, 58634fd. The tasks are: add a text `subtitle` field, add an image URL `coverImage` field, and repeat `subtitle`. The repository is reset to the scaffold between rounds, while StackWeft’s recipe store persists across rounds and prior runs: persistent skill memory is part of the method under test, and the run logs show recipe replay on most code slots in every round, so the StackWeft arm is measured in steady state. The baseline has no analogous store and starts cold each round. First-contact (cold) cost on a previously unseen repository is reported in the secondary-repository experiment.

Baseline. The baseline is a session-based agentic loop in the style of contemporary coding agents [Yang et al., 2024, Wang et al., 2024]: the same filesystem and shell tools, free exploration, and an append-only transcript fed back to the model at each step, implemented in our harness so that both arms share the same model gateway and per-call token

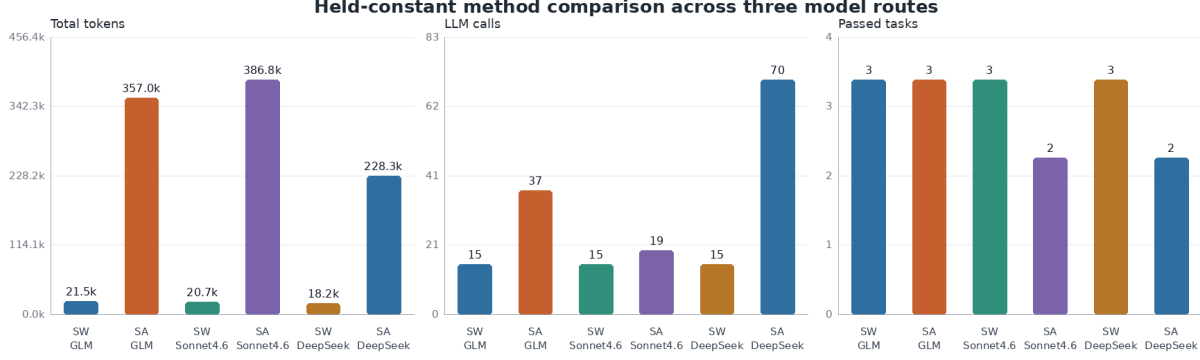


Figure 2: **Held-constant method comparison.** For each model route, both arms use the same model and the same sentinel scorer. StackWeft (SW) passes all tasks while using far fewer tokens; SA denotes the session-agent baseline, which fails one task under Claude Sonnet 4.6 and one under DeepSeek-V4-Pro.

Table 2: Held-constant comparison on three verified Conduit tasks per model route. LLM time sums successful model-call latency.

Model route	Method	Pass	Tokens	Calls	LLM time
GLM-5.1	StackWeft	3/3	21,460	15	274s
GLM-5.1	Session agent	3/3	356,979	37	540s
Claude Sonnet 4.6	StackWeft	3/3	20,661	15	185s
Claude Sonnet 4.6	Session agent	2/3	386,797	19	550s
DeepSeek-V4-Pro	StackWeft	3/3	18,156	15	200s
DeepSeek-V4-Pro	Session agent	2/3	228,287	70	697s

ledger. It receives the same requirement text and is instructed to deliver the field end-to-end. It has no Field Flow Graph, no recipes, and no slot evidence. The loop is capped at 24 tool rounds and a $\sim 120k$ -token budget per task. After it finishes, the same sentinel probe tests are injected and run.

Model control. The main comparison is repeated under three held-constant routes: GLM-5.1, Claude Sonnet 4.6 ([g] claude-sonnet-4-6-c), and DeepSeek-V4-Pro. In each run, both StackWeft and the session agent use the same model route for every generation call. We also report a StackWeft-only route robustness ablation on a single task (the subtitle text-field delivery) using GLM-5.1, Claude Sonnet 4.6, DeepSeek-V4-Pro, kimi-for-coding, gpt-5.5 via Codex routing, and MiniMax-M2.5-highspeed. For the first three routes, the ablation row reuses the first round of the main benchmark; the Kimi, Codex, and MiniMax routes were run separately with the same harness. Token accounting is taken from StackWeft’s per-call ledger and checked against the recorded gateway model IDs for each run.

Secondary repository. To test repository-shape generality, we run on a different React/Node/PostgreSQL CRUD project with `server/` and `webclient/` roots, object-form Sequelize fields, and `config/reducer` UI state. The shallow clone lacked a runnable test suite, so this experiment is scored by static cross-stack coverage over added diff lines: API model, API write path, and web render must all contain the new field.

5 Results

5.1 Main Conduit Benchmark

Table 2 and Figure 2 show the central result. Across three model routes and nine verified tasks, StackWeft passes 9/9 tasks; the session agent passes 7/9. StackWeft uses 60,277 total tokens, while the baseline uses 972,063, a $16.1\times$ reduction. The reduction holds under each held-constant route: $16.6\times$ for GLM-5.1, $18.7\times$ for Claude Sonnet 4.6, and $12.6\times$ for DeepSeek-V4-Pro. The comparison counts failed baseline attempts at their incurred token cost, because those tokens would still be billed in an interactive delivery session.

Table 3 decomposes the totals by round, which matters for interpreting the aggregate ratio. First, the reduction is not an artifact of the repeated round: every individual round, across all routes and both field kinds, shows a $10.8\text{--}24.1\times$

Table 3: Per-round decomposition of the main benchmark (total tokens). Ratio divides session-agent tokens by StackWeft tokens for the same round; failed rounds are charged at incurred cost. StackWeft passes every round; $\times$ marks the baseline’s failed rounds.

Model route	Round / field	StackWeft	Session agent	Ratio
GLM-5.1	1: subtitle	6,939	85,646	12.3 $\times$
	2: coverImage	7,505	131,813	17.6 $\times$
	3: subtitle (repeat)	7,016	139,520	19.9 $\times$
Claude Sonnet 4.6	1: subtitle	6,752	145,336	21.5 $\times$
	2: coverImage	6,728	162,190	24.1 $\times$
	3: subtitle (repeat)	7,181	79,271 $\times$	11.0 $\times$
DeepSeek-V4-Pro	1: subtitle	5,085	66,587	13.1 $\times$
	2: coverImage	6,913	95,478 $\times$	13.8 $\times$
	3: subtitle (repeat)	6,158	66,222	10.8 $\times$

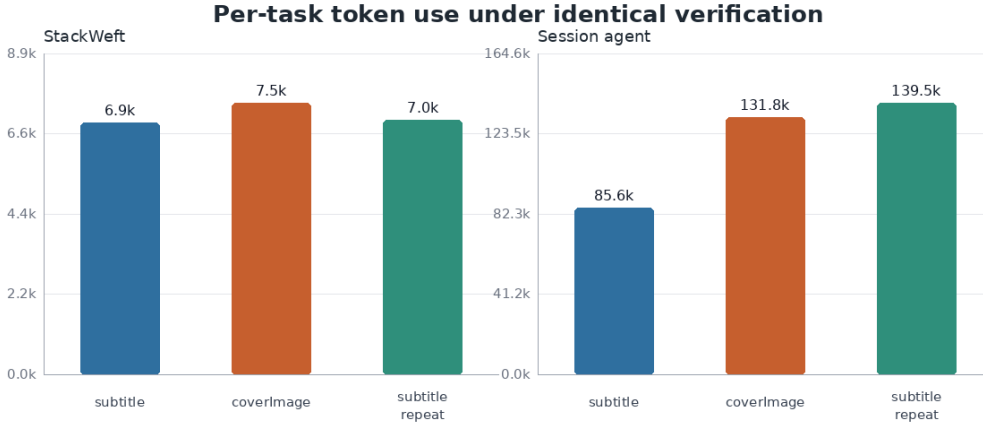


Figure 3: **Per-round GLM-5.1 tokens.** StackWeft remains near 7k tokens per task; the session agent repeatedly pays tens of thousands of input tokens even when the repository resets to the same scaffold.

reduction, and the two non-repeat rounds alone span 12.3–24.1 $\times$. Second, StackWeft’s per-round cost is nearly constant—5.1k to 7.5k tokens—because the billed surface is fixed by the slot contract rather than by exploration behavior; the run logs show recipe replay on most code slots in every round, so what the table measures is the steady-state cost: the fixed pipeline stages plus a small number of slot-local calls. The baseline’s per-round cost varies by 2.4 $\times$ across routes (66k–162k), and its failures are not cheap: the two failed rounds bill 79k and 95k tokens without delivering a verified change.

The token-time gap is informative. The session agent’s overhead is dominated by input tokens: it repeatedly re-sends long repository and interaction context. Wall-clock latency grows less dramatically than billed tokens because input prefill is comparatively cheap in time but not in billing; output decoding and round trips dominate latency.

5.2 Model Ablation

The ablation in Table 4 is not intended to rank models globally; it tests whether the compiled workflow is portable across heterogeneous APIs and response styles. All six routes complete the same `subtitle` task and pass the same verification gates.

5.3 Different-Repository Generality

On the secondary CRUD repository, both methods thread the field through all three checked layers. StackWeft’s first (cold) encounter with this repository costs 40,412 tokens over 14 calls—the price of profiling an unseen layout; the warm run, replaying the learned recipes, uses 5,578 tokens and 3 calls. The session agent uses 158,005 tokens and 9 calls, with no mechanism to amortize the discovery over later requests. This result is intentionally reported as static

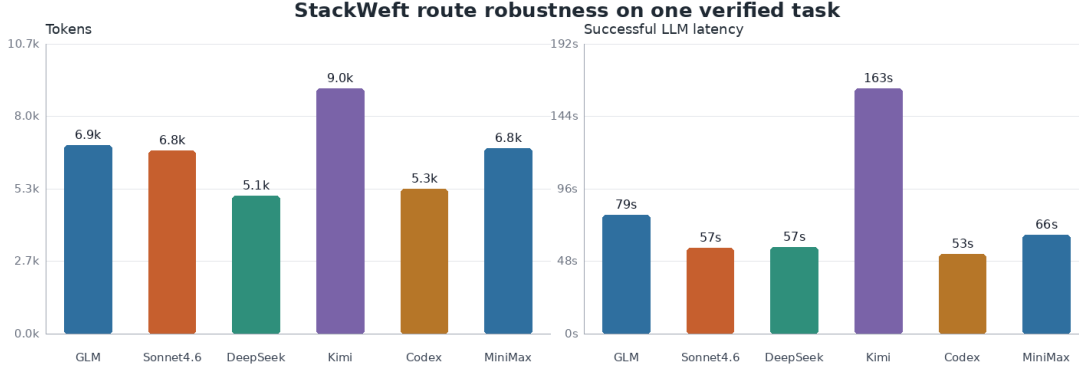


Figure 4: **StackWeft route robustness on a single task.** Six model routes pass the same verified `subtitle` delivery. The compiled slot contract keeps token use in a narrow band despite heterogeneous model APIs.

Table 4: Single-task StackWeft route robustness on the verified `subtitle` task. Calls include two failed embedding attempts logged by the system; all rows have three successful generation LLM calls.

Model route	Pass	Tokens	Calls	LLM latency
GLM-5.1	yes	6,939	5	79s
Claude Sonnet 4.6	yes	6,752	5	57s
DeepSeek-V4-Pro	yes	5,085	5	57s
Kimi for Coding	yes	9,040	5	163s
Codex gpt-5.5	yes	5,347	5	53s
MiniMax-M2.5-highspeed	yes	6,834	5	66s

coverage rather than green tests because the shallow clone lacks a runnable test harness. It nevertheless shows that Field Flow Graphs are not tied to the Conduit directory layout: the profiler adapts to `server/` and `webclient/` roots, object-form Sequelize models, and reducer-driven UI state.

6 Discussion

Where the gains come from. StackWeft’s advantage is not a stronger model. In the main experiments, the model route is held constant. The gains come from changing what the model sees: a slot-local patch problem rather than the full agent state. Recipe replay further converts repeated work into zero-LLM edits. In product terms, the system pays once to discover a workflow and then amortizes that discovery over future requests; this is the “use more, pay less” behavior that session-only coding agents do not naturally provide.

Why verification matters. A naive grep count can miss semantically important paths such as update handlers. StackWeft therefore supports fine-grained evidence declarations and sentinel probes. In practice, this caught an early failure where a field was accepted on create but not persisted on edit; the fix was to make profiled skills declare separate request, write-object, and update-assignment evidence.

Human interaction. StackWeft keeps explicit control points: clarification questions, confirm gates for risky requirements, pause/resume, and structured run inspection. The experiments in this paper use headless mode for reproducibility, but the same event log supports interactive intervention.

7 Limitations

The evaluation focuses on additive field-flow tasks, which are common in full-stack CRUD applications but do not cover arbitrary software engineering. Complex features such as new workflows, permission models, or algorithmic changes may require richer skills or a fallback general agent. The strongest correctness result is on Conduit, where runnable probes exist; the secondary repository result uses static coverage because the clone lacked tests. All benchmark figures come from single runs per configuration: we release exact per-call ledgers rather than means over repeated trials, so run-to-run variance is unquantified. Finally, the current profiler targets React/Express/Sequelize-like applications.

Other frameworks should be supported by adding profilers and skill templates rather than by weakening the verification contract.

8 Reproducibility

We retain the complete run records used to construct the tables and figures: per-call model route, token accounting, wall-clock latency, generated diff, and the verification outcome for each task. Each benchmark trial starts from the same repository revision, resets the target checkout before execution, and runs the same sentinel probes after generation. The released repository includes the benchmark harness and plotting code; reproducing the model-backed measurements requires configured gateway credentials for the corresponding model routes.

9 Conclusion

StackWeft demonstrates that a substantial class of full-stack software-delivery requests can be made cheaper and more reliable by compiling repository structure before generation. Field Flow Graphs turn a broad coding-agent problem into small, verifiable slot edits; recipe replay turns repeated patterns into deterministic work. On verified Conduit tasks, this design preserves pass rate while reducing tokens by $16.1\times$ across held-constant GLM, Claude Sonnet 4.6, and DeepSeek model routes, and the same workflow runs across GLM, Claude Sonnet 4.6, DeepSeek, Kimi, Codex, and MiniMax routes. The broader lesson is that software agents need not choose between raw autonomy and brittle scripts: for recurring product-delivery patterns, compiled, evidence-checked workflows offer a pragmatic middle ground.

References

- Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI), 2023.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Yoshihiko Futamura. Partial evaluation of computation process—an approach to a compiler-compiler. *Higher-Order and Symbolic Computation*, 12(4):381–391, 1999.
- Luca Gazzola, Daniela Micucci, and Leonardo Mariani. Automatic software repair: A survey. *IEEE Transactions on Software Engineering*, 2019.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven K. S. Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.
- Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.

- Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel Mankowitz, Esme Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- Fan Long and Martin Rinard. Automatic patch generation by learning correct code. In *ACM SIGPLAN Symposium on Principles of Programming Languages*, 2016.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Karl Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Yoann Padioleau, Julia Lawall, René Rydhof Hansen, and Gilles Muller. Documenting and automating collateral evolutions in Linux device drivers. In *European Conference on Computer Systems (EuroSys)*, 2008.
- RealWorld. Realworld example apps: The mother of all demo apps. <https://github.com/gothinkster/realworld>, 2026. Accessed 2026-06-11.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2006.
- TonyMckes. Conduit realworld example app. <https://github.com/TonyMckes/conduit-realworld-example-app>, 2026. React, Express, Sequelize monorepo used as the primary evaluation target.
- Rijnard van Tonder and Claire Le Goues. Lightweight multi-language syntax transformation with parser parser combinators. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019.
- Xingyao Wang et al. OpenHands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.

A Reproduction Protocol

The main benchmark uses three model routes and three additive field-delivery tasks. For each route, the harness runs StackWeft and the session-agent baseline from the same starting revision, applies the same natural-language requirement, and evaluates the resulting repository with the same probes. The route-robustness ablation runs the StackWeft arm only on the first text-field task and changes only the model route. The secondary-repository experiment follows the same protocol but uses static cross-stack coverage because that checkout does not provide a runnable test suite.

For every run, the harness records total tokens, number of model calls, model latency, pass/fail status, and the generated patch. Figures are regenerated from these logs without manual transcription.